



C-sync

SYNC EDUCATION WITH
REAL-WORLD TRENDS

FINAL PROJECT REPORT

AI Trend Agent

Keeping an AI engineering curriculum in sync with real-world technology change

A multi-agent system that monitors library releases and official blogs, verifies what it finds, locates the exact slides and lab cells each change affects, and recommends an evidence-backed curriculum update for an instructor to approve.

PREPARED BY

Abdurahman Al-Duraywish
TEAM LEAD

Aldanah Aldosari

Rafif Alsuhaibani

Ruyuf Almajnooni

REPORT GUIDE

Contents

	Executive summary	3
1	Introduction: background, problem, objectives and scope	4
2	System architecture	6
2.1	Overview	6
2.2	Data contracts	6
2.3	Design principles	7
2.4	Technology stack	7
3	Implementation	8
3.1	Monitoring	8
3.2	Clustering	8
3.3	Agent 1: Verification	9
3.4	Agent 2: Curriculum retrieval (RAG)	9
3.5	Agent 3: Evaluation	10
3.6	Agent 4: Recommendation	10
3.7	Instructor Companion and Fix Agent	11
3.8	Reproducibility: capture, replay and traces	11
4	The C-Sync interface	12
4.1	Home and Dashboard	12
4.2	Discover and Verify	14
4.3	Compare and Evaluate	16
4.4	Decide: the human in the loop	17
4.5	Ask and How it works	19
5	Results	21
5.1	The recorded run	21
5.2	Case study: langchain-core 1.6.4	22
5.3	What the run tells us	22
6	Evaluation	23
6.1	Offline test suites	23
6.2	Golden dataset and harness	23
6.3	Results	24
6.4	Evaluating the evaluator	25
7	Challenges and lessons learned	26
8	Limitations and future work	27
9	Conclusion	28
A-D	Appendices: team, running the system, repository, glossary	29

EXECUTIVE SUMMARY

An agent that recommends, and a person who decides

AI Trend Agent keeps an AI engineering course in step with the tools it teaches. It watches official release channels, verifies what it finds, pinpoints the slide or lab cell each change affects, and recommends a specific, cited action. An instructor approves every recommendation; the system never edits course material itself.

74SIGNALS
CAPTURED IN 30
DAYS**57**DISTINCT EVENTS
AFTER
CLUSTERING**29**RECOMMENDATIONS
WITH EVIDENCE**13**ACTIONABLE FOR
THE COURSE**257**OFFLINE TESTS, 0
API CALLS

The problem

Libraries such as LangChain and the OpenAI SDK publish releases every few days, while course material is revised between cohorts. In the 30 days before our final capture, the watched sources produced 74 signals, including 30 stable releases. Finding by hand which of the course's 107 files a release affects is slow and error-prone. A language model left to judge on its own gives confident answers even when a signal is fabricated, stale or irrelevant.

The approach

A deterministic clustering stage groups signals into events. Four agents then ask, in turn: is the event real, where is it taught, how much does it matter, and what should change? Models choose tool calls, search queries and wording; Python computes every score and action tier from facts the tools established. Failures are explicit, so a search that could not run is never reported as "no match". C-Sync, the interface, shows each recommendation as a traceable story from signal to decision and records the instructor's verdict.

Results

On the recorded run of 21 September 2026, the 30 largest of 57 events produced 29 recommendations: 6 to *update existing material*, 7 to *add optional content* and 16 to *keep watching*, each with its evidence and full agent trace. In the headline case, langchain-core 1.6.4 deprecated chat message history on the day of the capture. The system confirmed the release, followed the identifier `RunnableWithMessageHistory` to cell 36 of the Week 3 LangChain lab, and recommended updating that cell, with a score of 4.5 out of 5.

Quality

257 offline tests (243 for the pipeline and 14 for the verifier) run with zero API calls. A golden-dataset harness scores the pipeline layer by layer. Clustering scores 100 out of 100; verification scores 56.0 offline and 60.9 ± 3.4 live, for composite scores of 75.6 and 78.3 ± 1.9 . Building the harness also exposed a pairing bug that had misaligned earlier verification scores, and we fixed it.

HONEST LIMITS

The golden dataset is small (12 signals) and does not yet label the curriculum, evaluation and recommendation layers. The fix-drafting agent has not been exercised against a live model, and further live runs are blocked by the API project's spend limit. Section 8 sets out the next steps.

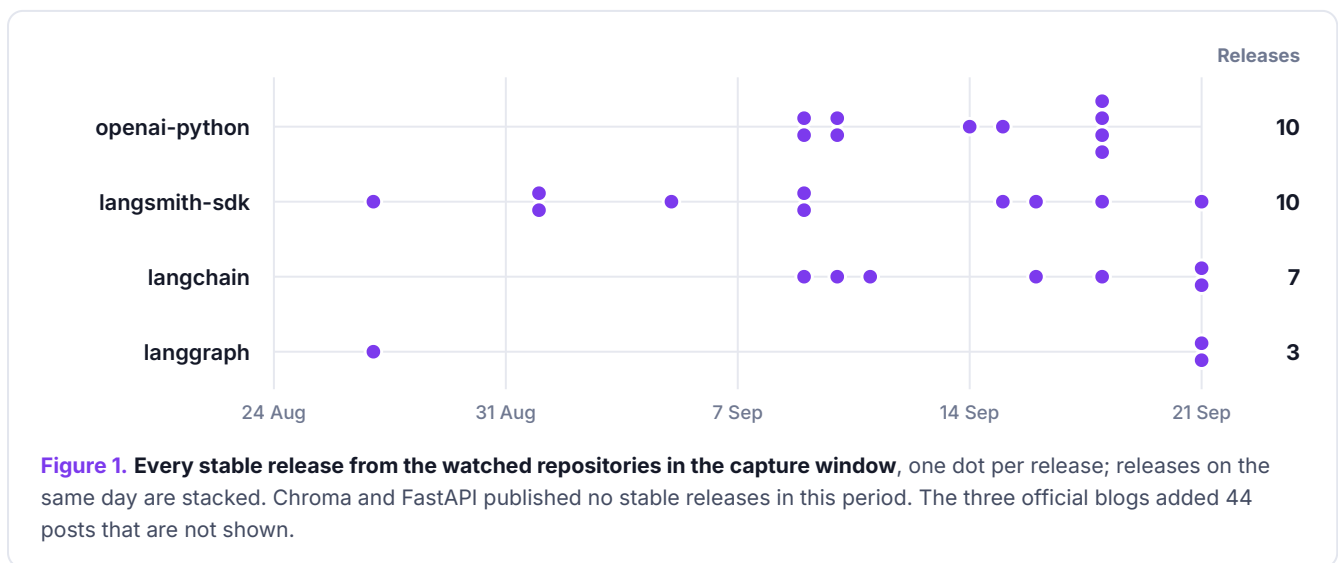
SECTION 1

Introduction

Course material and the tools it teaches change on different schedules. This project closes the gap between them without taking the decision away from the instructor.

1.1 Background and motivation

AI engineering is one of the fastest-moving areas of software. The frameworks taught in the WeCloudData Agentic AI Engineering program publish new versions every few days. They include LangChain, LangGraph, LangSmith, the OpenAI Python SDK, Chroma and FastAPI. In the four weeks before our final capture, four of the watched repositories published 30 stable releases (Figure 1), and three official vendor blogs published another 44 posts.



Course material changes on a slower clock. Slides and lab notebooks are written for a cohort and revised between cohorts. In the meantime, a release can deprecate an API that a lab calls, rename a module that a slide shows, or introduce a capability worth teaching. Students usually meet the problem first, as a deprecation warning or a failing cell in the middle of a lab. This is not hypothetical: our team ran into exactly this while taking the course.

1.2 Problem statement

Keeping a course current by hand means reading the release notes of every library it uses, deciding which changes matter, and then finding which of the 107 course files are affected. For Weeks 2 to 6, those files are 24 slide decks and 83 lab notebooks. The work is slow, easy to postpone and hard to do consistently. Automating it naively is also risky: a language model asked “does this release affect our course?” answers confidently even when the signal is fabricated, out of date or unrelated to what the course teaches.

PROBLEM STATEMENT

How can an instructor learn, soon after a technology change happens, exactly which slides or lab cells it affects and what to do about it, with evidence strong enough to trust and without the system changing the course on its own?

1.3 Objectives

1. **Monitor** primary sources, meaning official releases and official blogs, for changes relevant to the course.
2. **Consolidate and verify.** Group duplicate reports of the same event, then confirm with tools, rather than with model opinion, that each event is real.
3. **Locate** the affected course material precisely, down to the week, the file and the slide or notebook cell.
4. **Prioritise** each trend with transparent, reproducible scoring rules.
5. **Recommend** one of five action tiers with a concrete, cited plan, and leave the decision to an instructor.
6. **Explain and measure.** Make every recommendation traceable, and evaluate each stage against human labels.

1.4 Scope

In scope

- GitHub releases from six repositories and posts from three official blogs, with optional Hacker News feeds as secondary sources (Table 3).
- The WeCloudData curriculum for Weeks 2 to 6: slides and lab notebooks, indexed locally.
- Five action tiers, each with a cited action plan.
- An interface for review and approval, an instructor Q&A companion, and drafted fixes for lab cells.
- Offline tests and a golden-dataset evaluation harness.

Out of scope

- Editing course material automatically. The agent only recommends; a person approves.
- Continuous, scheduled monitoring. Runs are captured on demand and replayed.
- Sources beyond the configured feeds, such as package indexes and social media.
- Course content outside Weeks 2 to 6. The slide sources are kept outside the repository for redistribution reasons.

The rest of the report is organised as follows. Section 2 presents the architecture and Section 3 the implementation of each stage. Section 4 walks through the C-Sync interface, and Section 5 reports the results of the recorded run. Section 6 describes how we evaluated the system, and Sections 7 to 9 cover lessons learned, limitations and conclusions.

SECTION 2

System architecture

A linear pipeline of deterministic stages and tool-using agents, with a human at the end of every path.

2.1 Overview

Figure 2 shows the whole system. Monitoring collects signals from GitHub releases and official blogs, and clustering, which uses no language model, groups them into events. Four agents then run in sequence: **Verification** (is it real?), **Curriculum** (where is it taught?), **Evaluation** (how much does it matter?) and **Recommendation** (what should change?). C-Sync presents each recommendation with its evidence, and an instructor approves it, requests changes or rejects it. On demand, the **Instructor Companion** answers questions about a recommendation and the **Fix Agent** drafts the change to a lab cell.

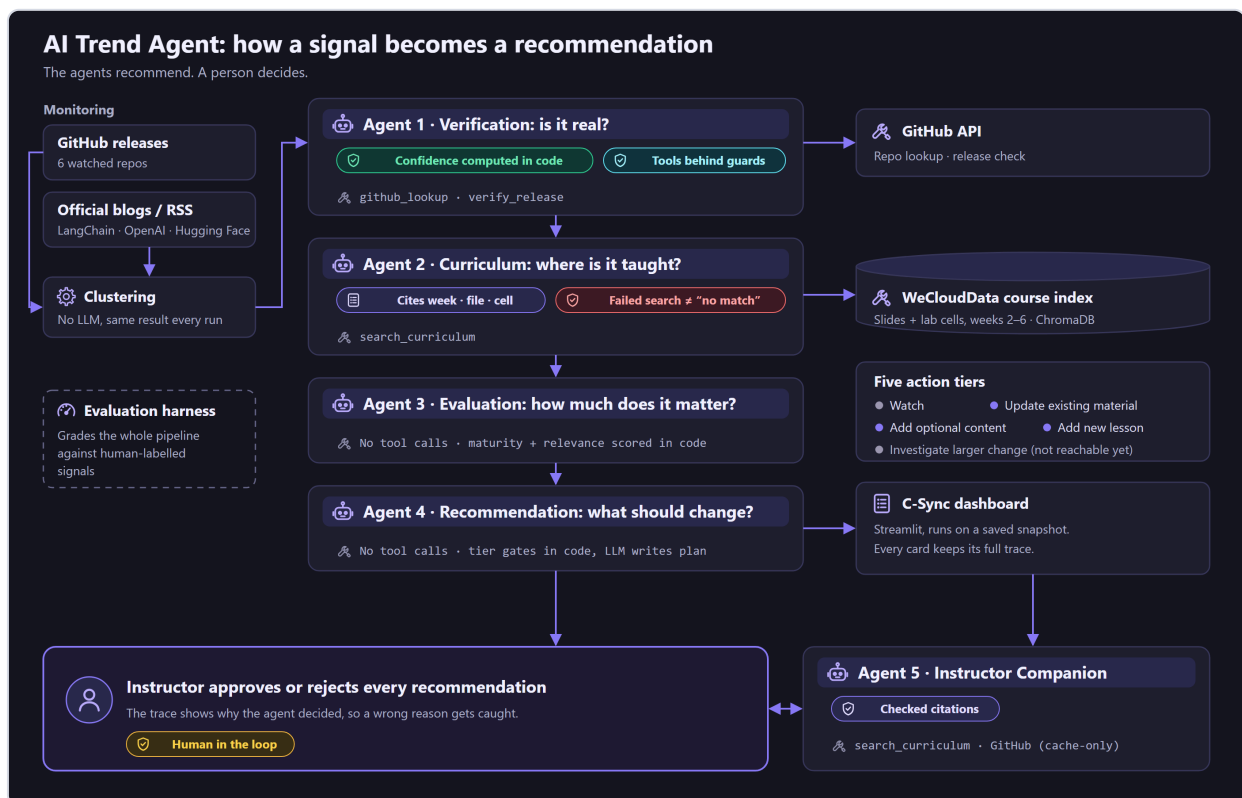


Figure 2. How a signal becomes a recommendation. The agents call tools (the GitHub API and the course index) behind guards, and every recommendation reaches an instructor. The evaluation harness grades the pipeline against human-labelled signals.

One rule divides the work throughout. Models make the judgements that need language: which tool to call next, which query to search, how to phrase a plan. Python computes every number and every decision that has consequences. Section 3 explains why.

2.2 Data contracts

Each stage's output is the next stage's input, and every contract is defined once, in `02_src/schemas.py`. Renaming a field requires telling the team, because downstream code would otherwise break silently. The contracts let the four workstreams build in parallel.

Table 1. Pipeline stages and the contract each one produces

STAGE	MODULE	OUTPUT	KEY FIELDS
Monitoring	<code>monitoring_github.py</code> , <code>monitoring_rss.py</code>	<code>RawSignal</code>	title, source, source tier (primary / secondary), summary, URL, published date
Clustering	<code>clustering.py</code>	<code>TrendCluster</code>	signals judged to report one event; independent source count
Agent 1: Verification	<code>agents/verification.py</code>	<code>VerifiedTrend</code>	confidence 0–1, status, evidence trail, reasoning steps, loop mode
Agent 2: Curriculum	<code>agents/curriculum.py</code>	<code>CurriculumMatch</code>	week, topic, file, slide or cell, similarity, exact identifier, content type
Agent 3: Evaluation	<code>agents/evaluation.py</code>	<code>EvaluationResult</code>	maturity 1–5, relevance 1–5, total score, rationale
Agent 4: Recommendation	<code>agents/recommendation.py</code>	<code>Recommendation</code>	action tier, action plan, evidence, match, total score

2.3 Design principles

- **Deterministic decisions, model-written language.** Scores and tiers are pure functions of checked facts: reproducible, testable offline, and impossible to argue upward.
- **Failures are states, not silence.** “Searched and found nothing”, “tried and failed” and “never searched” stay distinct.
- **Evidence travels with the recommendation.** Sources, tool results and reasoning steps come along, so a reviewer sees *why* as well as *what*.
- **Reproducible demonstrations.** A live run is captured once and replayed with zero API calls.
- **A human in the loop.** Nothing in the course changes until an instructor approves.

2.4 Technology stack

Table 2. Technologies used

Language	Python 3.10+
Language model	OpenAI <code>gpt-4o-mini</code> (set by <code>OPENAI_MODEL</code>), with function calling for every agent’s tool loop
Retrieval	ChromaDB with all-MiniLM-L6-v2 embeddings, plus a lexical pass fused by Reciprocal Rank Fusion
Document parsing	<code>python-pptx</code> (with speaker notes), <code>pdfplumber</code> (optional Tesseract OCR), notebook JSON
Sources	GitHub REST API (<code>requests</code>) and RSS/Atom feeds (<code>feedparser</code>)
Interface and quality	Streamlit (C-Sync); offline test suites, Streamlit <code>AppTest</code> and a custom evaluation harness

SECTION 3

Implementation

Each stage in turn: what it does, how it decides, and the measurements that shaped it.

3.1 Monitoring

`monitoring_github.py` reads the GitHub Releases API for six repositories and drops drafts and pre-releases (tags ending in `a`, `b` or `rc` plus a number), because students do not install those. `monitoring_rss.py` reads three official blogs. Both kinds are *primary* sources: the project's own announcement. Three Hacker News queries are available as *secondary* sources behind `--secondary`; they are noisy on purpose, to give the verifier something to filter out.

Table 3. Monitored sources and their signals in the recorded 30-day window (25 Aug – 21 Sep 2026)

SOURCE	TYPE	TIER	SIGNALS
<code>openai/openai-python</code>	GitHub releases	primary	10
<code>langchain-ai/langsmith-sdk</code>	GitHub releases	primary	10
<code>langchain-ai/langchain</code>	GitHub releases	primary	7
<code>langchain-ai/langgraph</code>	GitHub releases	primary	3
<code>chroma-core/chroma</code> , <code>fastapi/fastapi</code>	GitHub releases	primary	0
Hugging Face, OpenAI and LangChain blogs	RSS	primary	15 + 15 + 14
Hacker News: <i>langchain</i> , <i>AI agents</i> , <i>langgraph</i>	RSS (opt-in)	secondary	not used
Total			74

One operational lesson: the Hacker News feed provider sometimes returns malformed XML, and the fetcher then reports zero posts without an error. We therefore check the counts feed by feed after every capture, not only the total.

3.2 Clustering

Clustering does two jobs. It deduplicates, because a GitHub release and the blog post announcing it are one event and should not be verified or recommended twice. It also produces the *independent source count* that verification confidence depends on.

It runs in two passes, with no language model, so every run gives the same result. The first pass merges signals that share a rare identifier, such as `create_agent` or `AgentExecutor`, meaning one that appears in at most 8% of signals. The second pass falls back to title similarity with a 0.75 threshold. We raised the threshold from 0.4 after measuring on 63 real signals: at 0.4, unrelated posts merged on connective words and produced false high-confidence clusters. Maintainer handles such as `langchain-ai` sit on a stop list. In the recorded run, 74 signals formed 57 clusters, 8 of them with two or more signals.

3.3 Agent 1: Verification (“is it real?”)

The model drives a genuine tool-calling loop. It reads the cluster, decides which tool to call, judges whether each observation settles the question, and decides when to stop, within a cap of six rounds. `github_lookup` checks that a project exists (stars, last push), and `verify_release` confirms that a specific version actually shipped. We do not script the order of calls.

The model never produces a number. Python computes confidence from the facts that the *raw* tool results established: whether a matching repository exists, whether the specific claim was confirmed, and how many independent sources were corroborated. Hard ceilings are then applied.

Table 4. Deterministic verification confidence

CHECKED FACTS	CONFIDENCE
Two or more verified independent sources <i>and</i> the claim itself confirmed	0.95
Two or more verified sources, claim not confirmed	0.80
One verified source and the claim confirmed (for example, a release that <code>verify_release</code> found)	0.75
One verified source, or a first-party post with nothing independently checkable	0.60
Something real but nothing checkable, or a single unchecked secondary source	0.50 / 0.40
The named repository was not found	0.15
Ceilings applied afterwards: fewer than 2 verified sources → at most 0.75. Named repository missing → at most 0.30. Prompt-injection markers in the signal → at most 0.10. A “latest” claim superseded by a newer release, or a publisher who is not the release author → 0.0 and status <i>contradicted</i> .	

Guards protect the facts that the scorer relies on:

- **Repository matching.** Each lookup result is checked against the repository the signal names. A lookup for “langchain” once returned `microsoft/markitdown` first, because results are sorted by stars, and the top result was taken as confirmation.
- **“Unchecked” is not “missing”.** A network error or cache miss is recorded as unchecked, never as “repository not found”. A repository can be declared missing only if a signal names it, because the model sometimes invents repositories.
- **Deterministic fallback.** Without an API key, or if the model fails, the same tools run in a scripted loop and the same scorer applies, so a score means the same thing in either mode. The trace records which mode ran.

WHY CODE, NOT THE MODEL, COMPUTES CONFIDENCE

When an earlier version let the model report confidence directly, it returned 0.85 for a single-source trend *and* 0.85 for a two-source trend, although the rubric separated the two cases. Models compress ranges; code does not. A deterministic score is also testable offline, and a persuasive model cannot talk it upward.

3.4 Agent 2: Curriculum retrieval (“where is it taught?”)

Indexing. `curriculum_ingest.py` extracts text from slide decks (`.pptx` with speaker notes; `.pdf` with optional OCR) and from lab notebooks, cell by cell. The week comes from the folder name and the topic from the file name. Each chunk records whether it is a *slide* or a *lab* cell and has a stable ID, so re-ingestion updates

chunks instead of duplicating them. The index holds **2,721 chunks from 107 files** covering Weeks 2 to 6.

Hybrid search. Embeddings blur rare identifiers. On the real decks, a slide that literally contains “FAISS” scored 0.303 against a FAISS query, which is indistinguishable from unrelated noise at 0.31. Search therefore fuses semantic and lexical ranks with Reciprocal Rank Fusion ($k = 60$) and flags any *exact identifier* match. The semantic relevance floor of 0.48 was calibrated on the course decks: genuine matches scored 0.65–0.66, while unrelated text scored around 0.31.

The agent. Similarity search always returns *something*, so something has to judge the results. The agent writes its own queries, preferring identifiers such as `RunnableWithMessageHistory` to prose, and can search again within a limit of four steps. It must also decide whether a hit is actually affected. Its prompt defines impact strictly: a trend affects an item only when it makes the material incorrect, outdated, incompatible, misleading or broken, and shared vocabulary does not count. It checks labs as well as slides, because a notebook cell that calls a deprecated API stops running.

THREE OUTCOMES THAT MUST NEVER BE MERGED

Searched: the agent looked, and its reason says what it found. **Search failed:** it tried, but the search could not run, which is *not* a finding of “no match”. **Skipped:** verification confidence was below 0.4, so no search was attempted. Collapsing these states was the most serious bug we found (Section 7).

3.5 Agent 3: Evaluation (“how much does it matter?”)

Python computes two scores from values the pipeline already trusts, and the model only explains them in plain language. If the model’s reply states or argues for a number, it is rejected, and a template built from the same facts is used instead. The rationale therefore cannot contradict the score.

Table 5. Scoring bands (total = $\frac{1}{2}$ maturity + $\frac{1}{2}$ relevance)

SCORE	QUESTION	RULE
Maturity 1–5	Is this real and established?	From verification confidence: $\geq 0.85 \rightarrow 5$; $\geq 0.70 \rightarrow 4$; $\geq 0.50 \rightarrow 3$; $\geq 0.30 \rightarrow 2$; otherwise 1.
Relevance 1–5	How directly does it hit what we teach?	Exact identifier match $\rightarrow 5$. Slides: similarity $\geq 0.65 \rightarrow 5$, $\geq 0.55 \rightarrow 4$, $\geq 0.48 \rightarrow 3$. Labs: $\geq 0.55 \rightarrow 5$, $\geq 0.45 \rightarrow 4$. A weak match floors at 2; no match at all $\rightarrow 1$.

Labs get the lower thresholds on purpose, because a broken lab cell is more urgent than an outdated concept slide. “No match” (1) is kept separate from “weak match” (2), because the recommendation stage treats the absence of coverage as a distinct state.

3.6 Agent 4: Recommendation (“what should change?”)

Python picks the tier, because a curriculum lead acts on it, so it must be reproducible and testable. The first rule that applies wins:

Table 6. Deterministic tier selection

#	CONDITION	TIER
1	Maturity below 4: not established enough to act on	KEEP WATCHING
2	A curriculum match exists and relevance ≥ 4	UPDATE EXISTING MATERIAL

#	CONDITION	TIER
3	A match exists with lower relevance	ADD OPTIONAL CONTENT
4	No match, and the curriculum was never checked (skipped or failed)	KEEP WATCHING
5	No match, but the trend is a version bump (e.g. <code>v3.9.0</code> , <code>==1.6.2</code>)	KEEP WATCHING
6	No match, and the title names nothing the course could teach	KEEP WATCHING
7	Verified, in scope and genuinely uncovered	CREATE A NEW LESSON

The model writes the plan (at most five steps and 1,500 characters), and the plan is then checked against the tier. A plan whose wording argues for a different tier, such as “new lesson” under *keep watching*, is rejected, and so is one that contains instruction-override phrases. This stops a prompt-injected recommendation from slipping through in the prose even when the tier itself is safe. Releases that cite the same lab cell are collapsed into one recommendation. The fifth tier, *investigate larger change*, is deliberately unreachable: an evaluation carries a single match, so there is no multi-module signal to base it on, and inventing one would be fabrication.

3.7 Instructor Companion and Fix Agent

Instructor Companion

A conversational layer over *one* recorded recommendation. The companion answers questions such as “why this tier?” or “which lab cell is hit?” from the record and its trace. It explains the decision and never recomputes or overrules it. Follow-up lookups use the local course index and cached GitHub responses only, so they never make a live GitHub request. Answers cite typed labels: [R] record, [E] evidence, [M] match, [C] curriculum searches, [V] verification steps and [T] chat lookups. A checker flags invented labels and claims that contradict the trace.

Fix Agent

For an *update existing material* recommendation, the Fix Agent drafts the change itself. It retrieves the full lab cell from the index, together with the release evidence, and proposes a before-and-after version of the cell. It may change only what the evidence supports, must keep the lab’s `# YOUR CODE HERE` placeholders, and must list anything the evidence does not settle. Code then flags every identifier in the draft that appears in neither the cell nor the evidence. The draft is labelled *not applied* and is never written anywhere.

3.8 Reproducibility: capture, replay and traces

The live pipeline is measurably non-deterministic: across three live runs over the same 10 signals, 5 of the 10 changed tier at least once. We document that variance and demonstrate from a frozen run instead.

`demo_snapshot.py --capture` runs the pipeline once and saves every recommendation with its agent trace. C-Sync and `--replay` read that file with zero API calls. A trace records every verification step (the thought, the tool and the observed result), every curriculum query, and whether the curriculum was searched, failed or skipped. Successful GitHub responses are cached on disk, and `TOOL_CACHE_ONLY=1` runs the tools without touching the network.

SECTION 4

The C-Sync interface

A story-led Streamlit interface that follows one recorded trend from signal to decision, in the order a reviewer needs to see it.

C-Sync reads the recorded run and never re-runs an agent, so every page agrees with every other. Only two actions can call a model, and both need an explicit click: asking the companion a question, and pressing *Draft the fix*. The left panel lists all nine pages. The top bar holds the five pipeline stages, *01 Discover* to *05 Decide*, and each stage opens its page. The interface builds on the team's SkillRadar UI prototype.

All screenshots were captured from the recorded run of 21 September 2026 (`01_data/demo_snapshot.json`) with the featured trend, langchain-core 1.6.4, selected. No review decisions had been recorded at capture time.

4.1 Home and Dashboard

The Home page opens with the big picture. Each square is one stage of the run, and its area is proportional to the real count at that stage, so the reader sees how much noise the pipeline removes before it recommends anything. A featured trend invites the reader to follow one decision from beginning to end.

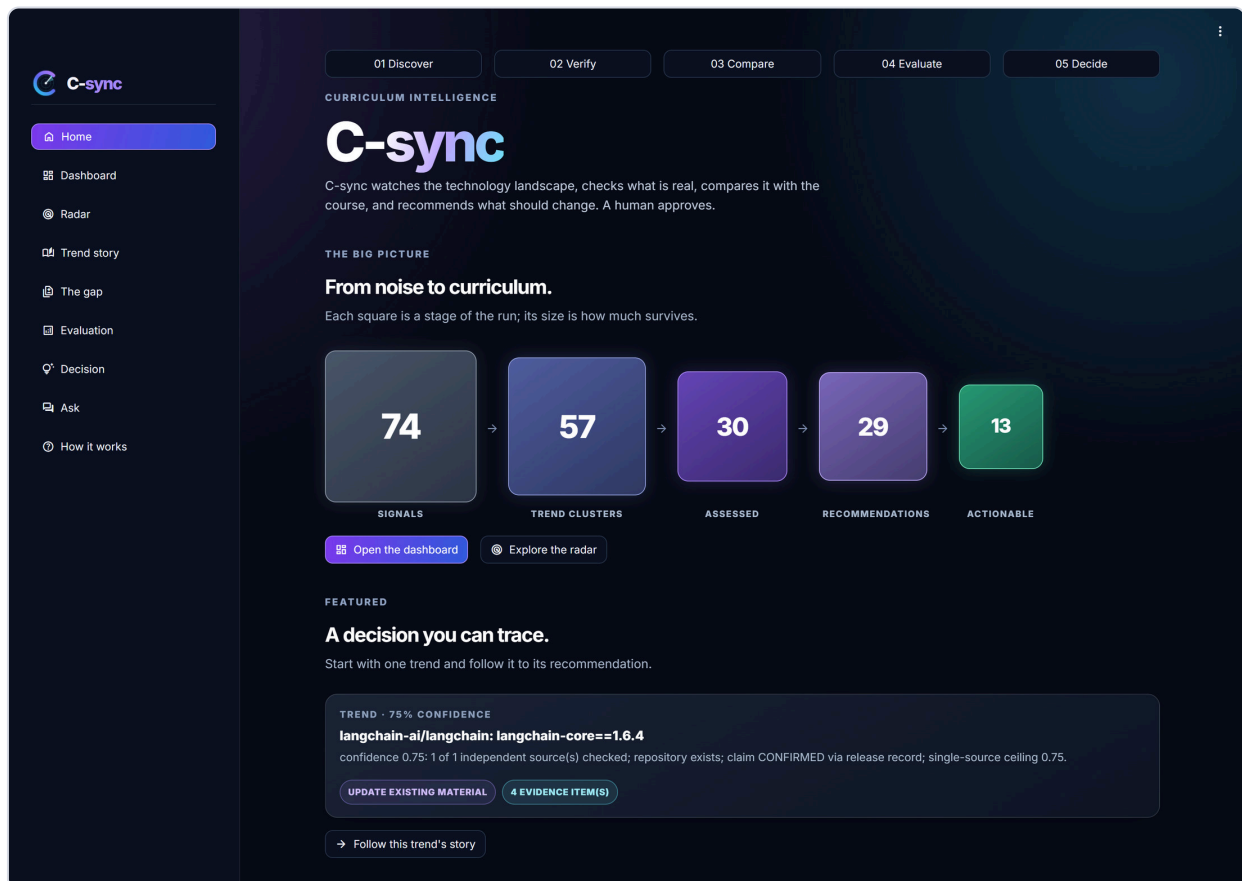


Figure 3. Home. “From noise to curriculum” sizes each stage by its real count: 74 signals → 57 clusters → 30 assessed → 29 recommendations → 13 actionable. The left panel lists every page; the top bar holds the five pipeline stages.

The Dashboard lists every recommendation, most urgent tier first. It can be filtered by action tier, by the kind of material cited (lab or slides) and by "actionable only". Each card shows the score, the confidence, the exact course citation and the instructor's review status. The action plan sits behind a "Full plan" expander, so that the list stays easy to scan.

C-sync

Home Dashboard Radar Trend story The gap Evaluation Decision Ask How it works

01 Discover 02 Verify 03 Compare 04 Evaluate 05 Decide

ALL RECOMMENDATIONS

Dashboard

Every recommendation in the run, most urgent first.

Action: Update existing material x Add optional content x Keep watching x

Material: All Lab Slides Actionable only

29 of 29 recommendations

langchain-ai/langchain: langchain-core==1.6.4 4.5 / 5

AWAITING INSTRUCTOR REVIEW

UPDATE EXISTING MATERIAL 0.75 CONFIDENCE

LAB

Week 3 / Lab: Lab LangChain wikipedia(studentVersion) / cell 36

→ Story Decision

> Full plan

openai/openai-python: v3.16.0 4.5 / 5

AWAITING INSTRUCTOR REVIEW

UPDATE EXISTING MATERIAL 0.75 CONFIDENCE

SLIDES

Week 4 / Agents for Real Tasks.pptx / slide 53

→ Story Decision

> Full plan

openai/openai-python: v3.12.0 4.5 / 5

AWAITING INSTRUCTOR REVIEW

UPDATE EXISTING MATERIAL 0.75 CONFIDENCE

LAB

Week 2 / Lab: Story Writing with Prompt Chaining OpenAI / cell 6

→ Story Decision

> Full plan

openai/openai-python: v3.10.0 4.5 / 5

AWAITING INSTRUCTOR REVIEW

UPDATE EXISTING MATERIAL 0.75 CONFIDENCE

LAB

Week 3 / Lab: Demo LangChain Document Chat using OpenAI(solution) / cell 17

→ Story Decision

> Full plan

langchain-ai/langchain: langchain-openai==1.6.3 4.0 / 5

AWAITING INSTRUCTOR REVIEW

UPDATE EXISTING MATERIAL 0.75 CONFIDENCE

LAB

Week 5 / Lab: Demo Diagnosing Coordination Failures(Solution) / cell 4

→ Story Decision

> Full plan

openai/openai-python: v3.13.0 4.0 / 5

AWAITING INSTRUCTOR REVIEW

UPDATE EXISTING MATERIAL 0.75 CONFIDENCE

SLIDES

Week 4 / Agents for Real Tasks.pptx / slide 44

→ Story Decision

> Full plan

Figure 4. Dashboard (the first three rows of 29 cards). The highest-scoring recommendations each point at a specific course location, such as *Week 3 / Lab: Lab LangChain wikipedia (studentVersion) / cell 36*.

4.2 Discover and Verify

The Radar (01 Discover) shows each assessed trend as a light on a sweeping scanner. Colour encodes the recommended action, size the maturity score and glow the verification confidence. Clicking a light opens that trend's story without reloading the app.

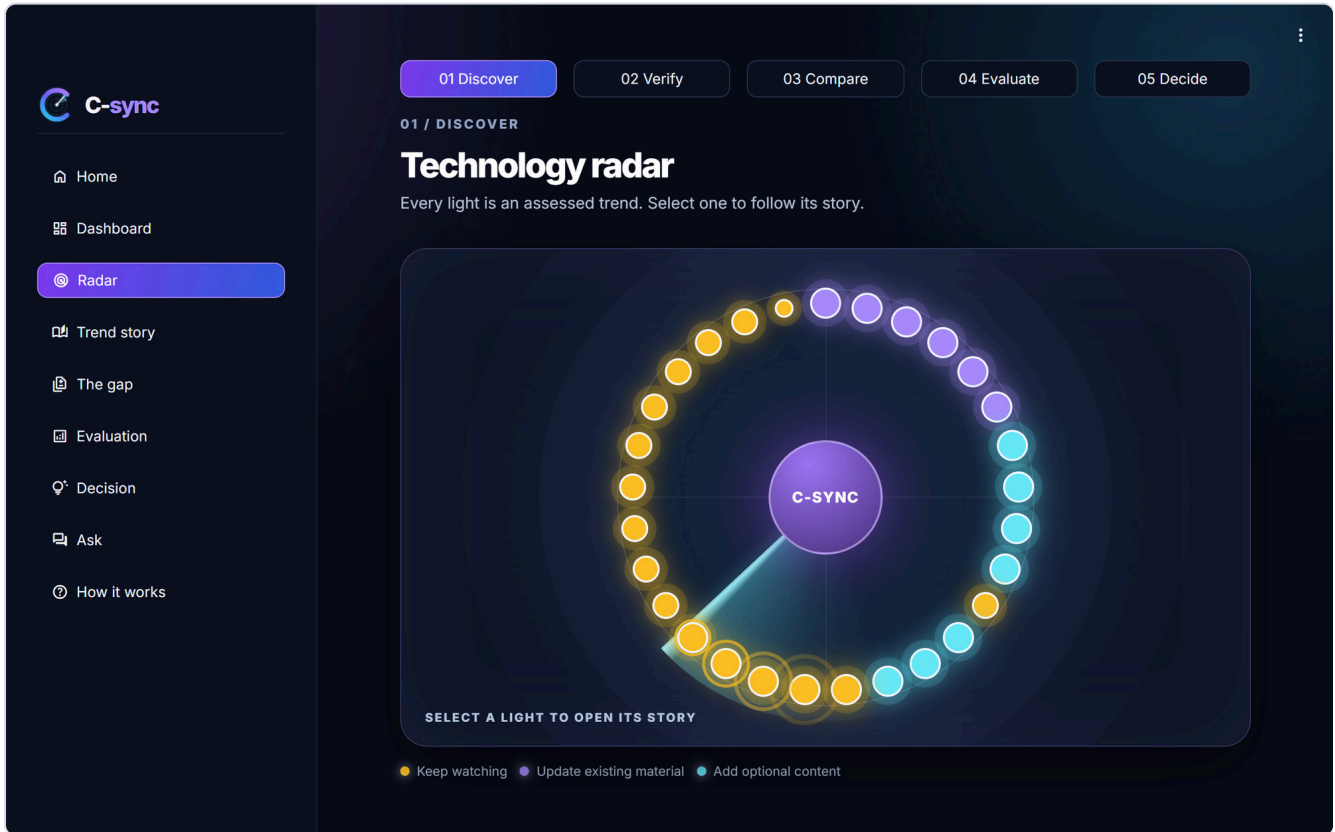


Figure 5. Radar (01 Discover). 29 lights, one per recommendation: amber is *keep watching*, violet *update existing material* and cyan *add optional content*. A light flashes as the beam passes over it.

DESIGN CHOICES THAT RUN THROUGH EVERY PAGE

- **Story order.** Each stage page leads to the next with a button, so a reviewer reads the evidence before the verdict.
- **Only recorded facts.** Repository stars come only from the recorded lookup. A release check turns green only when `verify_release` confirmed the release, and a failed curriculum search is shown as failed.
- **Accessible by default.** Visible focus rings, a primary colour that gives white text a 5.7:1 contrast ratio, reduced-motion support, and a phone layout with a collapsing sidebar.

The Trend story (02 Verify, Figure 6) answers “why is this trending, and why should we trust it?”. It shows the original signal, including its release notes, then the evidence the Verification Agent recorded, one card at a time.

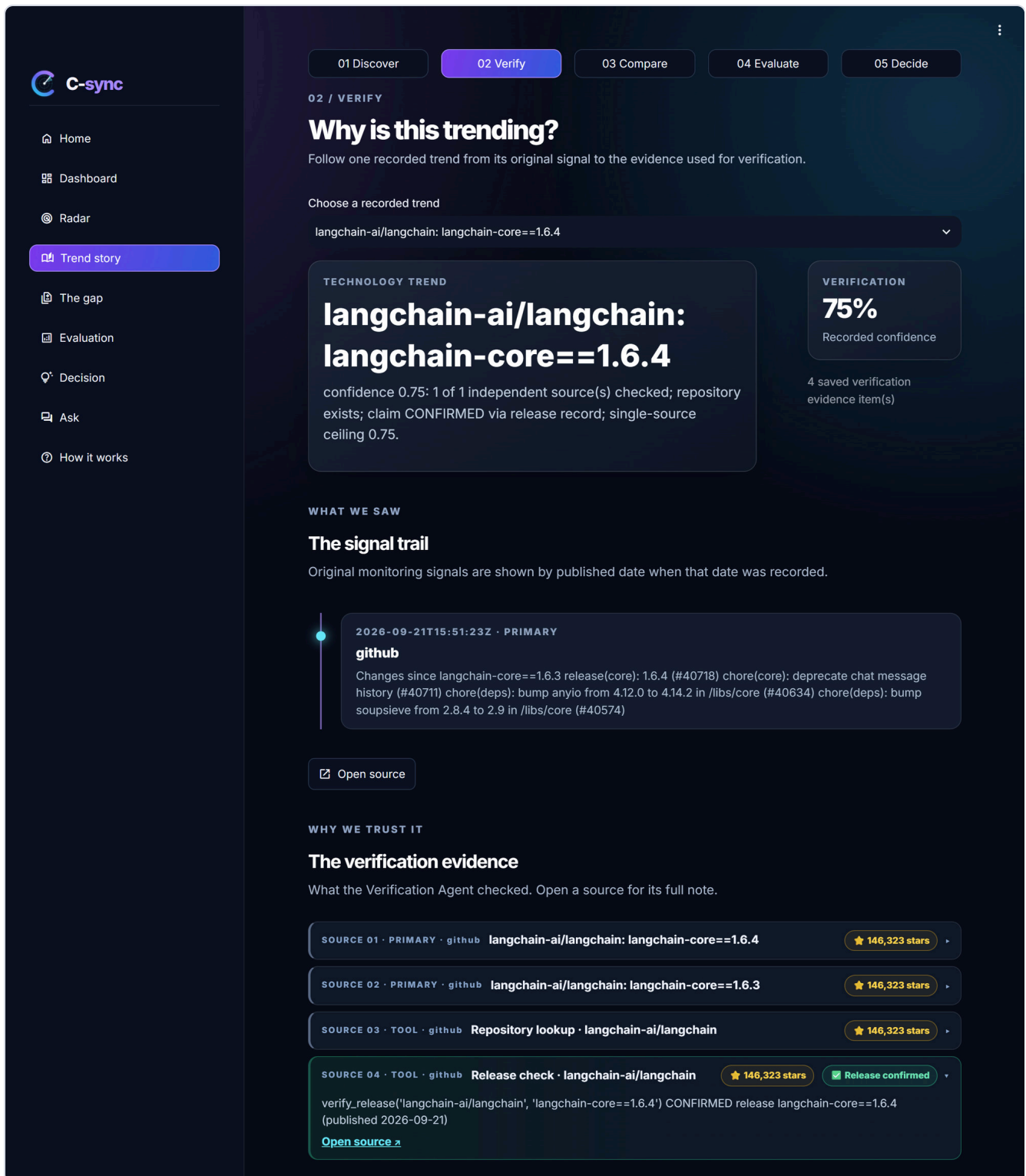


Figure 6. Trend story (02 Verify) for langchain-core 1.6.4. The release notes include “deprecate chat message history”. The evidence includes the repository lookup (146,323 stars) and the confirmed release check, which is expanded here.

4.3 Compare and Evaluate

The gap (03 Compare) places the trend beside the course material that the Curriculum Agent matched. The verdict is phrased with care. “Related material found” needs a citation, and “Curriculum gap identified” appears only when the agent actually proposed a new lesson. Otherwise the page says “Coverage not established”, because a missing match alone does not prove a gap.

C-sync

- Home
- Dashboard
- Radar
- Trend story
- The gap**
- Evaluation
- Decision
- Ask
- How it works

01 Discover 02 Verify **03 Compare** 04 Evaluate 05 Decide

03 / COMPARE

Is this trend already taught?

Place the recorded trend beside the course material the Curriculum Agent found.

Choose a recorded trend

langchain-ai/langchain: langchain-core==1.6.4

TECHNOLOGY TREND

langchain-ai/langchain: langchain-core==1.6.4

confidence 0.75: 1 of 1 independent source(s) checked; repository exists; claim CONFIRMED via release record; single-source ceiling 0.75.

CURRENT COURSE MATERIAL

Week 3 / Lab: Lab LangChain wikipedia(studentVersion) / cell 36

[Code cell] # If we want to keep track of these messages automatically, we can use the following code: #
https://python.langchain.com/docs/expression_language/how_to/message_history from langchain_community.chat_message_histories import ChatMessageHistory from langchain_core.runnables.history import RunnableWithMessageHistory

Related material found

A saved course citation links this trend to existing material. The recorded action explains what should change.

MATCH DETAIL

The specific course reference

Inspect the citation used by the backend.

MATCHED COURSE MATERIAL

Week 3 / Lab: Lab LangChain wikipedia(studentVersion) / cell 36

Lab LangChain wikipedia(studentVersion)

LAB EXACT IDENTIFIER: RUNNABLEWITHMESSAGEHISTORY SIMILARITY 0.502

Read the cited course content

Lab_LangChain_wikipedia(studentVersion).ipynb

```
[Code cell]
# If we want to keep track of these messages automatically, we can use the following code:
# https://python.langchain.com/docs/expression_language/how_to/message_history
from langchain_community.chat_message_histories import ChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

message_history = # YOUR CODE HERE,
endgame_agent_with_chat_history = R
```

Figure 7. The gap (03 Compare). The trend beside Week 3, LangChain lab, cell 36. The match was made through the exact identifier `RunnableWithMessageHistory`, and the cell's code, which imports `ChatMessageHistory`, is open for inspection.

Evaluation (04 Evaluate) keeps the focus on the evidence until the reader asks for the numbers. When *Reveal the scores* is pressed, three rings show maturity, relevance and the overall score, and a caption explains how each was derived from the stored run.

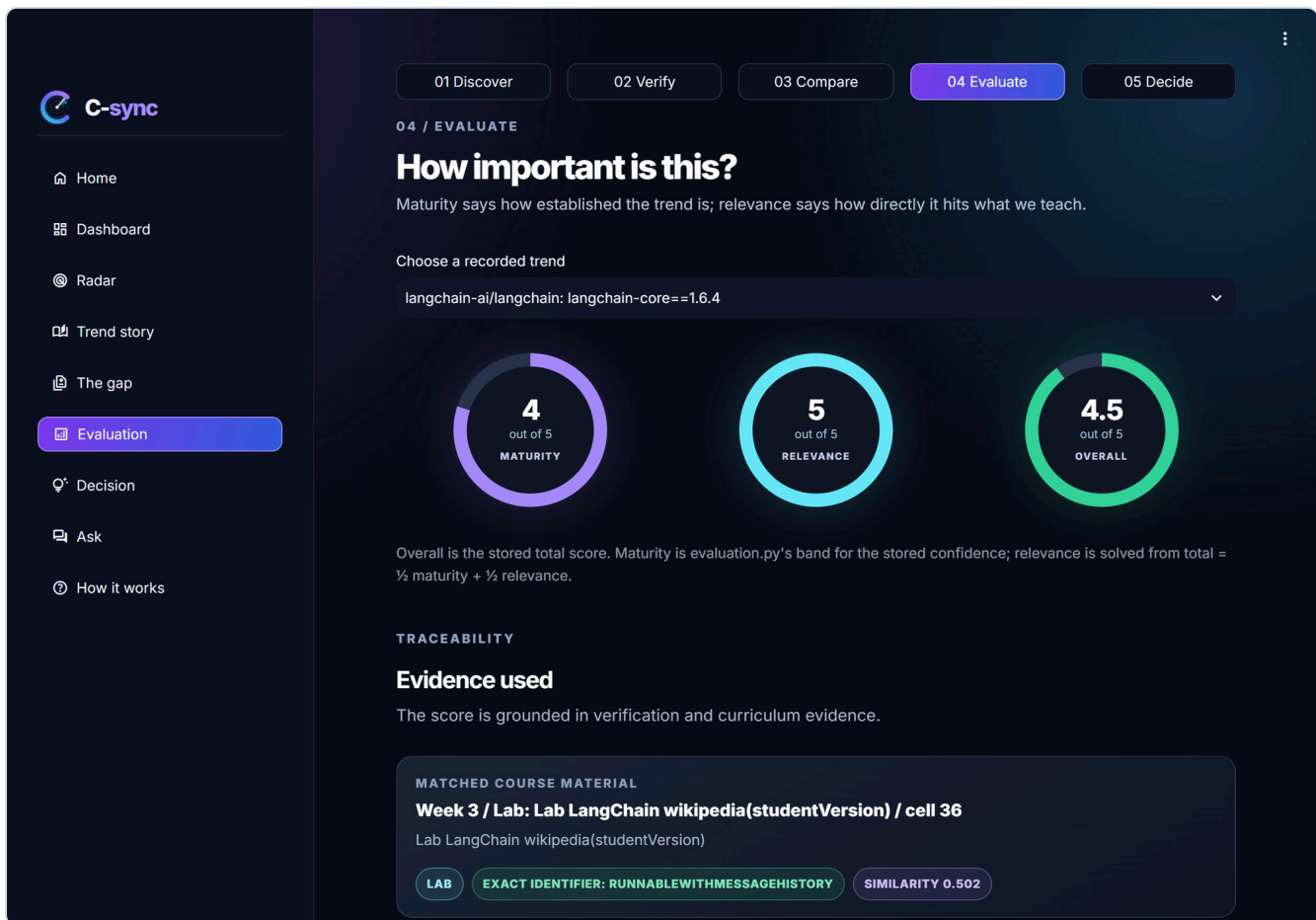


Figure 8. Evaluation (04 Evaluate). Maturity 4 (confidence 0.75), relevance 5 (an exact identifier match in a lab) and an overall 4.5 out of 5, with the matched course material shown as the evidence behind the scores.

4.4 Decide: the human in the loop

The Decision page (05 Decide) is where the system hands over to a person. It shows the recommended action with its verification note, review status and cited course area, then the plan the Recommendation Agent wrote (Figure 9). Below the plan, *The proposed fix* drafts a before-and-after version of the lab cell. The Fix Agent runs only when the instructor presses *Draft the fix*, and the draft is never applied.

Your decision (Figure 10) records the verdict (Approve, Request changes or Reject) with an optional note, and can attach the drafted fix. The verdict then appears as a status pill on the Decision header and on the Dashboard card. A collapsible evidence chain shows the path from trend to evidence, curriculum, evaluation and decision on one line.

C-sync

- Home
- Dashboard
- Radar
- Trend story
- The gap
- Evaluation
- Decision**
- Ask
- How it works

01 Discover 02 Verify 03 Compare 04 Evaluate **05 Decide**

05 / DECIDE

What should we do?

A specific curriculum action, with its reason and supporting evidence close at hand.

Choose a recorded trend

langchain-ai/langchain: langchain-core==1.6.4

RECOMMENDATION · LANGCHAIN-AI/LANGCHAIN: LANGCHAIN-CORE==1.6.4

Update existing material

confidence 0.75: 1 of 1 independent source(s) checked; repository exists; claim CONFIRMED via release record; single-source ceiling 0.75.

AWAITING INSTRUCTOR REVIEW 4 EVIDENCE ITEM(S)

WEEK 3 / LAB: LAB LANGCHAIN WIKIPEDIA (STUDENT VERSION) / CELL 36

WHAT CHANGES

The action plan

Steps returned by the saved Recommendation Agent run.

- 01 update the import to the new module path in cell 36 of the Lab LangChain notebook to reflect the latest version of the library.
- 02 ensure that any references to `langchain\_core` are updated to match the new module structure as per the latest documentation.
- 03 review the code in the cell for any deprecated functions or methods that may need to be replaced with their current equivalents.
- 04 test the updated code to confirm that it runs successfully without errors after the changes are made.

THE SOLUTION

The proposed fix

The change itself, drafted from the full course cell and the release evidence. Nothing is applied until an instructor approves it.

[Draft the fix](#)

Figure 9. Decision (05 Decide), part 1. *Update existing material* for Week 3 lab cell 36, marked *awaiting instructor review*, with the four-step action plan and the *Draft the fix* button.

HUMAN IN THE LOOP

Your decision

C-sync only recommends. Nothing in the course changes until an instructor approves.

Awaiting instructor review

Review the plan and the proposed fix, then decide.

Note for the team (optional)

e.g. Approved; update cell 36 before the next cohort.

☒ Approve ☐ Request changes ☐ Reject

Why this decision? · Evidence chain

TREND
langchain-ai/langchain: langchain-core==1.6.4

→

EVIDENCE
4 saved item(s)

→

CURRICULUM
Week 3 / Lab: Lab LangChain wikipedia(studentVersion) / cell 36

→

EVALUATION
Saved total 4.5/5

→

DECISION
Update existing material

☒ Inspect verification evidence

Figure 10. Decision, part 2 (content area). The review panel and the evidence chain from trend to decision. C-sync only recommends; nothing in the course changes until an instructor approves.

4.5 Ask and How it works

The Ask page puts the Instructor Companion in front of one recommendation. Pills show the tier and the curriculum outcome (searched, search *failed*, skipped or no trace), so the three states are never blurred. Suggested questions help an instructor start. Without an API key, the page restates what the run recorded instead of answering.

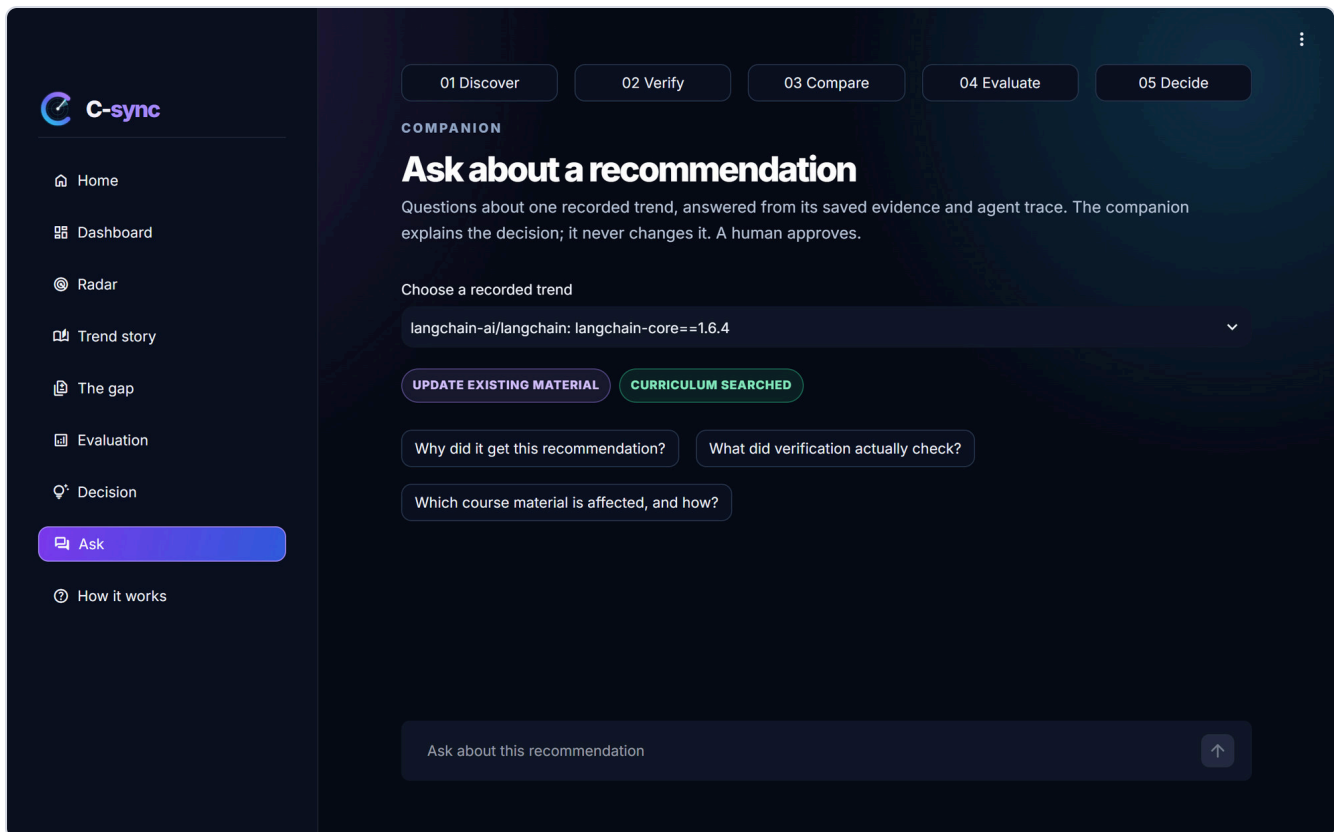


Figure 11. Ask. The companion answers questions about one recorded recommendation from its evidence and trace, using typed, checked citations. It explains the decision and cannot change it.

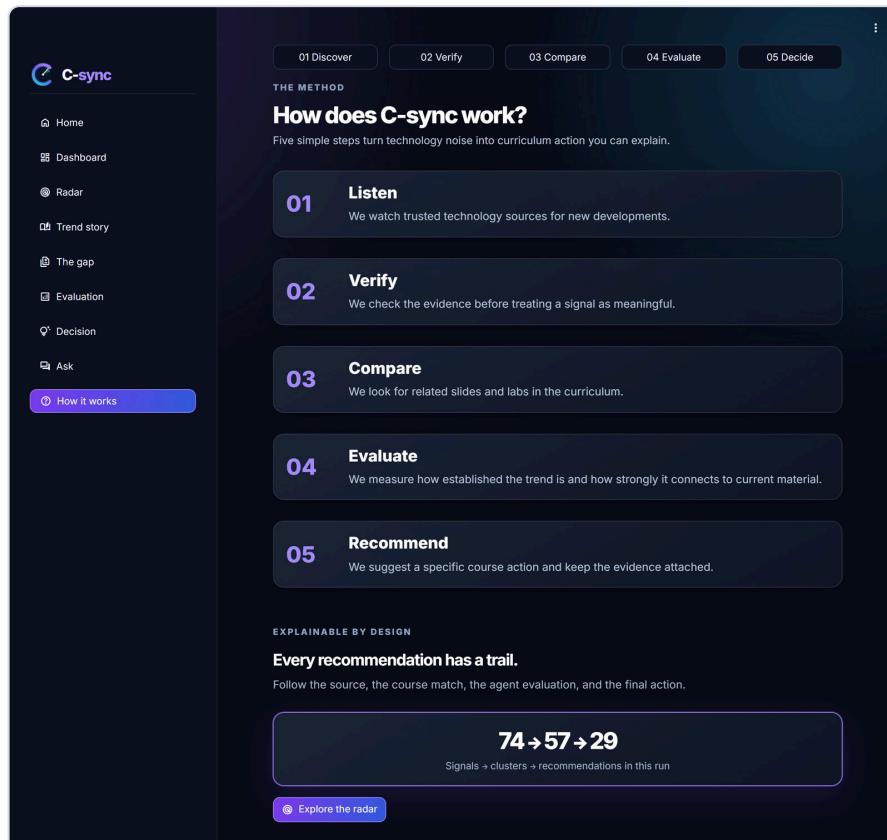


Figure 12. How it works. The five steps in plain language for readers new to the system, with the run's signals → clusters → recommendations trail.

SECTION 5

Results

What the system produced on real data: a 30-day capture of the course's own toolchain, recorded on 21 September 2026.

5.1 The recorded run

The capture fetched 30 days of primary-source signals: 74 signals, which clustering reduced to 57 events. The pipeline assessed the 30 largest clusters and collapsed one pair that cited the same material, leaving 29 recommendations. Every recommendation carries a trace. The Verification Agent ran in agentic mode on all 29. The curriculum was searched for 28 trends and skipped once, below the confidence gate, and not a single search failed.

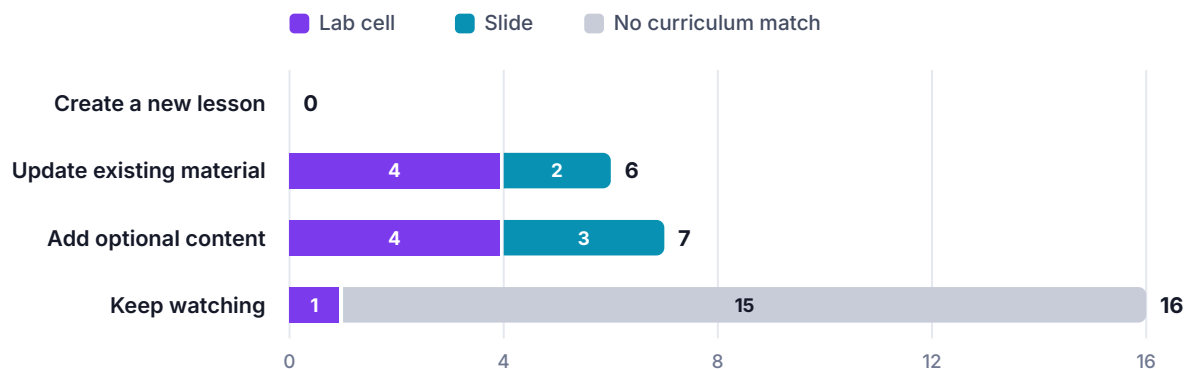


Figure 13. Recommendations by action tier and by the kind of course material they cite. Every actionable recommendation cites a specific slide or lab cell, and 8 of the 13 cite a lab cell. No trend qualified as *create a new lesson* (Section 5.3).

Table 7. The six “update existing material” recommendations

RELEASE	CITED COURSE MATERIAL	MATCHED BY	SCORE
<code>langchain-core==1.6.4</code>	Week 3 / Lab: Lab LangChain wikipedia (studentVersion) / cell 36	identifier <code>RunnableWithMessageHistory</code>	4.5
<code>openai-python v3.16.0</code>	Week 4 / Agents for Real Tasks / slide 53	identifier <code>MCP</code>	4.5
<code>openai-python v3.12.0</code>	Week 2 / Lab: Story Writing with Prompt Chaining (OpenAI) / cell 6	similarity 0.685	4.5
<code>openai-python v3.10.0</code>	Week 3 / Lab: Demo LangChain Document Chat using OpenAI (solution) / cell 17	similarity 0.734	4.5
<code>langchain-openai==1.6.3</code>	Week 5 / Lab: Demo Diagnosing Coordination Failures (solution) / cell 4	similarity 0.595	4.0
<code>openai-python v3.13.0</code>	Week 4 / Agents for Real Tasks / slide 44	similarity 0.639	4.0

5.2 Case study: langchain-core 1.6.4

The headline recommendation shows every stage at work on the kind of change that motivated the project: a release that deprecates an API used by a student lab.

SIGNAL · GITHUB RELEASE · 21 SEP 2026

`langchain-ai/langchain: langchain-core==1.6.4`. The release notes list *"chore(core): deprecate chat message history (#40711)"*.

VERIFICATION · AGENTIC LOOP · 2 TOOL CALLS

`github_lookup` matched `langchain-ai/langchain` (146,323 stars), and `verify_release` **confirmed** the release, published 21 Sep and not a pre-release. Confidence is **0.75**: one independent source with its claim confirmed, at the single-source ceiling.

CURRICULUM · 2 QUERIES

"chat message history" ranked Week 3 lab cell 36 first (0.585). A follow-up query for `RunnableWithMessageHistory` found that identifier verbatim in three cells. The agent's reason: the lab teaches `RunnableWithMessageHistory` and `ChatMessageHistory`, which the deprecation makes potentially outdated.

EVALUATION

Maturity **4** (confidence ≥ 0.70) and relevance **5** (exact identifier in a lab cell) give **4.5 / 5**.

RECOMMENDATION

UPDATE EXISTING MATERIAL: a match is present and relevance is at least 4. The plan: update the imports in cell 36, align references to `langchain_core` with the current documentation, replace deprecated calls, and re-run the cell.

HUMAN REVIEW

Shown in C-Sync as *awaiting instructor review*. The instructor can draft the fix, then approve it, request changes or reject it.

Figures 6 to 8 show this same trace in C-Sync. Because the evidence and the reasoning travel with the recommendation, a reviewer can check each step instead of trusting the conclusion.

5.3 What the run tells us

- **Confidence follows the evidence, not the prose.** The 18 GitHub-release trends score 0.75: one independent source with its claim confirmed. The 10 blog-post trends score 0.60, as first-party posts with nothing independently checkable, which puts them at maturity 3, below the action floor.
- **No false "new lessons".** Five releases matched nothing in the course. Each was a version bump, so each correctly stayed at *keep watching*, instead of prompting a lesson about a library the course already teaches (Section 7).
- **Labs dominate the actionable list.** 8 of the 13 actionable recommendations cite a lab cell. That reflects both the scoring, which treats labs as more urgent, and the reality that code breaks before concepts do.
- **A new lesson needs corroboration.** *Create a new lesson* requires a verified, in-scope trend with no match that is not a version bump, which in practice means a new capability confirmed by a release check or by two independent sources. This capture had none. The rule is conservative by design (Section 8).
- **One known artefact.** A genuine blog post scored 0.15 because the model invented a repository to look up. A guard now prevents this (Section 3.3), but the recorded run is kept as captured.

SECTION 6

Evaluation

Three tiers of evidence: deterministic tests on every change, a human-labelled golden dataset scored layer by layer, and behavioural cases aimed at failures we actually observed.

Offline test suites exercise the deterministic logic with zero API calls. A golden dataset of human-labelled signals, including fabricated claims, is scored layer by layer, with repeated runs to average out model non-determinism. Behavioural test cases target specific observed failures. We wrote twelve such cases for promptfoo but could not run them, because the tool needs Node 22.22 or later (Section 8).

6.1 Offline test suites

`test_chain.py` blanks the API key when it is imported, so a real key in `.env` is never used. It runs **243 checks, all passing**. `test_verification.py` adds **14 passing tests** of the verifier's tool loop. Both suites run before every merge.

Table 8. What the offline suite guards (sections of `test_chain.py`)

AREA	GUARANTEES TESTED
Verification (sections 1, 2, 10, 13)	Scoring bands and ceilings; repository matching; injection, staleness and publisher gates; the restored deterministic verifier and its four real-data fixes
Evaluation and recommendation (3, 4, 12)	Score mapping; every tier rule; a fallback verdict can never be acted on
Retrieval and tools (6, 7)	The FAISS case (an exact identifier beats similarity); tools report errors as data and never raise into the loop
Safety (5)	Prompt-injection markers in titles, summaries and slide text
Failure states (9, 11)	A curriculum search failure driven through the real agent loop, and end to end through <code>capture()</code>
Observability (8, 14)	Agent traces are recorded, and their plain-language display keeps searched, failed and skipped apart
Interface (16)	Every C-Sync page is smoke-run with Streamlit <code>AppTest</code> ; only the Ask page and the fix panel may import an agent
Companion, fix and review (17, 18)	Citation checks, offline and error modes, placeholder preservation, flagging of new names, persistence of review decisions

6.2 Golden dataset and harness

The golden dataset holds 12 signals labelled by hand: 10 genuine and 2 fabricated. Each label records whether the claim is genuine, the confidence verification *should* assign, whether the signal presents stale news as new, a priority rank, the expected maturity and relevance, and the expected action tier. The labelling guide asks reviewers to leave a field empty rather than guess, because a guessed label silently invalidates the score.

`run_eval.py` scores five layers, and `compare.py` refuses to compare runs that used different datasets (by SHA-256 hash) or different models.

Table 9. Metrics by pipeline layer

LAYER	METRICS	WEIGHT	STATUS
Clustering	Contamination rate; cluster count against the expected count	0.20	SCORED
Verification	Truth–confidence gap (genuine minus fabricated); confidence MAE; stale-flag rate	0.25	SCORED stale flag needs a new output field
Curriculum	Precision@3; recall; correct abstention	0.15	NULL needs event-level citation labels
Evaluation	Spearman rank correlation; maturity and relevance MAE	0.20	NULL needs event-level labels
Recommendation	Exact tier accuracy; off-by-one accuracy; over-recommendation rate	0.20	NULL needs event-level tier labels

A layer whose labels or outputs are missing scores *null*, never zero, and the composite renormalises its weights over the available layers. On the current dataset the composite is therefore a weighted mean of clustering and verification. The verification score is the mean of the truth–confidence gap $\times 100$ and $(1 - \text{MAE}) \times 100$. Each run records the commit, dataset hash, model, token usage and the mean $\pm$ standard deviation over its repeats.

6.3 Results

We scored four configurations of the verifier on the same dataset (SHA-256 `eb13e5ada377...`). The three offline runs used cached tool responses and no model (`TOOL_CACHE_ONLY=1`). The live run used `gpt-4o-mini-2024-07-18` with three repeats. Clustering scored 100 in every run: zero contamination, and 11 clusters against the 11 expected.

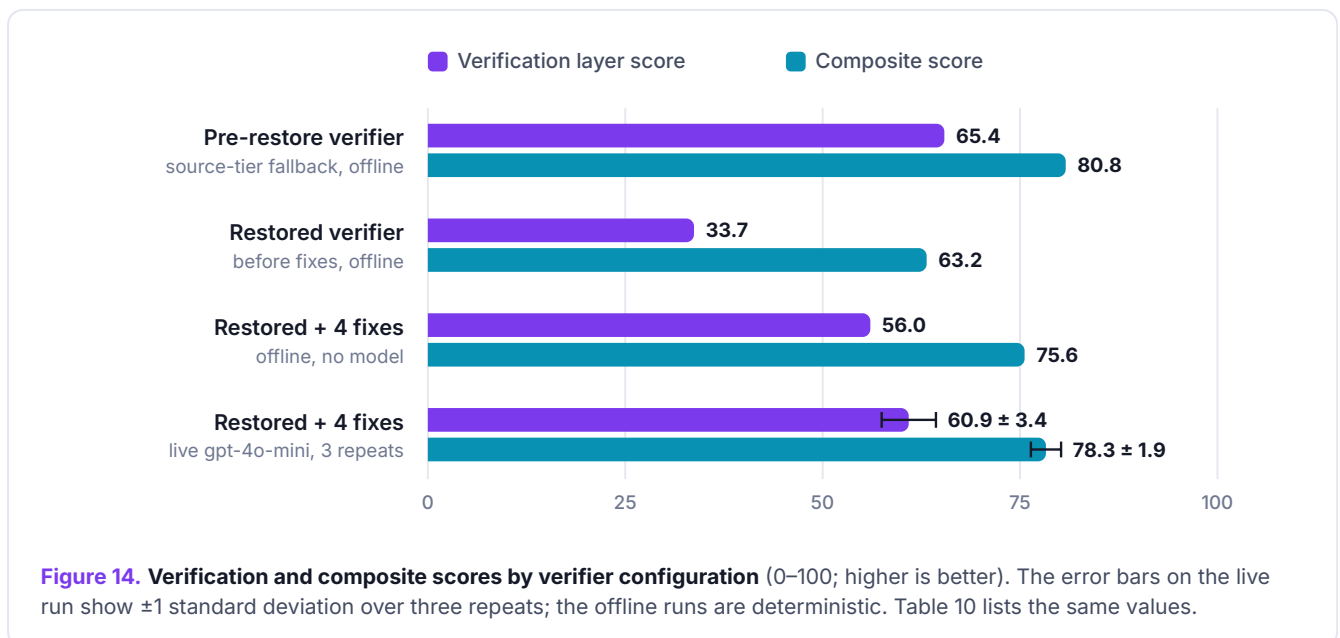


Table 10. Evaluation results on the golden dataset (12 signals, 3 repeats each)

VERIFIER CONFIGURATION	MODE	VERIFICATION	GAP	MAE	COMPOSITE
Pre-restore, source-tier fallback 3f5c2a7	offline	65.42	0.450	0.142	80.79
Restored deterministic verifier, before fixes 7d435e8	offline	33.71	-0.005	0.321	63.17
Restored verifier + 4 real-data fixes c4c9355	offline	56.04	0.275	0.154	75.58
Restored verifier + fixes, live model 156d86f	live ×3	60.90 ± 3.44	0.358	0.140	78.28 ± 1.91

The four fixes came from running the restored verifier on real data. Monorepo tags such as `langchain==1.4.0` are confirmed verbatim. A bare-name “not found” counts only against claims that rest on secondary sources alone, and a bare-name match never confirms such a claim, because of name-squatting repositories. A first-party post with nothing checkable scores 0.60, below the action floor. Together, the fixes raised the verification score from 33.7 to 56.0 offline. The live run, in which the model chooses the tool calls, scored 60.9 ± 3.4 and used about 245,000 tokens over its three repeats.

READING THESE NUMBERS HONESTLY

In this 12-item set, every fabricated item is a secondary source and every genuine item a primary one, so a rule that looks only at source tier separates them by construction. The pre-restore verifier’s higher score reflects that shortcut, not better verification. The restored verifier scores only what its tools confirmed, which is the behaviour we want on real data, where source tier and truth do not line up so neatly. The dataset needs harder cases before verification scores can be compared with confidence.

6.4 Evaluating the evaluator

The harness itself had a bug, and finding it changed our numbers. `run_eval.py` used to pair trends with labels using `zip(trends, entries)`, that is, one trend per *cluster* against one label per *signal*. The default set has 11 clusters and 12 signals, so after the first merged cluster every trend was scored against the *next* signal’s label. Each signal is now matched to the cluster that contains it. Every verification score produced before the fix, including figures we had previously reported, was misaligned and has been retired. A second fix made the dataset hash independent of line endings, so a Windows checkout of the same labels no longer looks like a different dataset to `compare.py`. The lesson: evaluation code deserves the same scrutiny as the code it evaluates.

SECTION 7

Challenges and lessons learned

Most of the design in Section 3 exists because something went wrong on real data first. These are the failures that taught us the most.

7.1 A failed search looked exactly like “no match”

The Curriculum Agent returned nothing both when it searched and found no match *and* when the search itself failed. During one live run the API spend limit was hit (HTTP 429). Every search failed silently, and three trends received confident *create a new lesson* recommendations claiming “no existing coverage found”, although no search had run. **Fix:** a `search_failed` state with a reason on every failure path; a rule that keeps unchecked trends at *keep watching*; and a banner stating that a failed search is *not* a finding of “no match”. The fix is verified offline end to end, and the re-run at scale is waiting for API quota.

7.2 Models compress confidence, and code can go missing

Model-reported confidence did not separate one source from two, so all scoring moved into code (Section 3.3). A version-control lesson came with it: the deterministic verifier had never been committed, so for a while the model set confidence. We restored it from the team’s reference copy and re-scored it (Table 10).

7.3 Title similarity merged unrelated news

Clustering by title alone produced spurious cross-source merges that looked like corroborated, high-confidence events. An identifier-first pass with a strict 0.75 title fallback fixed this (Section 3.2).

7.4 “Create a new lesson” for everything

In one live batch, nine of ten trends became new-lesson recommendations. They included independent journalism in Ukraine and the Navier–Stokes Millennium Prize problem: all verified, all uncovered, none in scope. An in-domain gate now requires a positive technical signal in the *title*, because vendor names appear in the body of every post a vendor publishes.

7.5 A version bump is not a curriculum gap

openai-python v3.7.0 to v3.10.0 each proposed a new lesson introducing the OpenAI Python library, which the course already teaches in several labs. A release that matches nothing now stays at *keep watching*.

7.6 Tools return the wrong thing, and models invent inputs

A GitHub search for “langchain” returned `microsoft/markitdown` first, because results are sorted by stars, and the top result was taken as confirmation. The model also invented `openai/openai` while checking a genuine post. Lookups are now matched against the repository a signal names, and only a named repository can be reported as missing.

7.7 Matching words is not matching meaning

“Announcing LangGraph v0.1” matched a cell that already used the new API, where the old name appeared only in a comment. “An Alien Mind” matched an AI-ethics slide through the word “alignment”, but the essay argued policy while the slide taught technique. Exact matching cannot tell code from commentary about code. That is why a human approves every recommendation, and why labelled positive cases are scarce.

7.8 Live runs are not repeatable

The same 10 signals changed tier on 5 of 10 across three live runs. We capture a run once, replay it deterministically and document the variance, rather than demonstrating live and hoping.

SECTION 8

Limitations and future work

What the current system cannot yet claim, and the work that would change that.

Limitations

- **The evaluation covers only part of the pipeline.** The default golden dataset has 12 signals, and its tier labels are 10 × *keep watching* and 2 × *add optional content*, with no *update existing material* or *create a new lesson* labels. A 13-signal variant adds one *update* case (LangChain and LangGraph v1.0, `create_react_agent` → `create_agent`), but it lacks the fields that the clustering metrics need. The curriculum, evaluation and recommendation layers score null until the labels are event-level.
- **Two metrics need new outputs, not more labels.** The stale-flag rate needs an explicit staleness field on `VerifiedTrend`, and precision@3 needs the Curriculum Agent to expose its top three candidates rather than only its final choice.
- **Behavioural tests have not been run.** The twelve promptfoo cases are written, but promptfoo requires Node 22.22 or later and the development machine runs 21.6.
- **Some agents are lightly exercised.** The Instructor Companion has been tested live only lightly, and the quality of its answers has not been evaluated. The Fix Agent's first live request hit the spend limit; the interface correctly showed the error and a retry button, but no draft has been produced live yet.
- **Live runs are blocked.** The OpenAI project's spend limit prevents the at-scale re-run that would confirm the search-failure fix, and a full evaluation re-run. The limit applies to the project, so a new key does not help.
- **The design is conservative.** Blog-only trends cannot reach an action tier without corroboration, and *investigate larger change* is unreachable. Both choices trade recall for trust.
- **Retrieval has a narrow range.** MiniLM compresses similarity scores into a narrow band. The thresholds are calibrated to it and must be re-measured if the embedding model changes.

Future work

1. **A Content-Type Agent.** Classify each signal as a release, announcement, case study, self-promotion or opinion before the tier gates, to filter the Show HN posts and vendor case studies that pass the in-domain gate.
2. **An event-level golden dataset with positive cases.** Merge the two dataset variants, key labels by event, and add verified *update* and *new lesson* cases. That would unblock a whole column of metrics at once.
3. **Scheduled monitoring and notifications.** Run the capture on a schedule and notify instructors about new actionable recommendations.
4. **Multi-module signals.** Let an evaluation carry several matches, so that *investigate larger change* becomes reachable for paradigm shifts.
5. **From approval to pull request.** Turn an approved fix into a pull request against the course repository, so that a human still reviews and merges it.
6. **More sources and stronger retrieval.** Add package indexes and documentation changelogs, and evaluate a larger embedding model against the calibrated thresholds.

SECTION 9

Conclusion

AI Trend Agent shows that an agentic system can keep a fast-moving course current without asking anyone to trust a black box.

We set out to tell an instructor, soon after a change happens, exactly which slides or lab cells it affects and what to do about it, with evidence strong enough to trust. On a real 30-day capture of the course's own toolchain, the system reduced 74 signals to 29 evidence-backed recommendations. It pinpointed 13 actionable changes to specific slides and lab cells, including the deprecation that motivated the project, which it traced from a GitHub release to cell 36 of the Week 3 LangChain lab.

The most important decisions were about *where* to use a language model. Models choose tool calls, write queries and explain decisions; code computes confidence, scores and tiers from facts the tools established. Failures are explicit states rather than silent defaults, every recommendation carries its trace, and an instructor makes the final call. Each of these rules exists because we saw what happened without it on real data.

The evaluation is honest about its reach. The deterministic core is well tested, with 257 offline tests. The golden dataset scores clustering and verification, but it needs event-level labels and harder cases before it can grade the later stages. With those labels, a Content-Type Agent and scheduled runs, C-Sync could move from a recorded demonstration to a tool that instructors use every week.

The agent recommends. A human decides. Every recommendation shows its work.

APPENDICES

Appendices

A Project team

NAME	ROLE
Abdurahman Al-Duraywish	Team Lead · integration: end-to-end pipeline, C-sync interface, Docker
Rafif Alsuhaibani	Verification agent and its tests · gold-dataset evaluation
Aldanah Aldosari	Evaluation agent · SkillRadar UI, the base of C-sync
Ruyuf Almajnooni	Data ingestion · curriculum retrieval (hybrid RAG search) and the curriculum agent

The work was organised into four workstreams: Data & Ingestion, Agents & Tools, Platform & Security, and Evaluation & QA. Capstone project for the SDA × WeCloudData Agentic AI Engineering Program. Repository: github.com/DHM22/AI-Trend-Agent, branch [feature/ai-trend-agent](#).

B Running the system

Python 3.10+, `pip install -r requirements.txt`, and copy `.env.example` to `.env`. `OPENAI_API_KEY` is needed only for live agent runs; `GITHUB_TOKEN` raises the GitHub rate limit. On Windows, set `PYTHONIOENCODING=utf-8` before printing feed titles.

```
# the interface: replays the recorded run, no API calls except Ask and "Draft the fix"
python -m streamlit run c_sync/app.py

# offline checks (0 API calls); run both before merging
python 02_src/agents/test_chain.py
python 02_src/tests/test_verification.py

# replay or capture a run
python 02_src/demo_snapshot.py --replay
python 02_src/demo_snapshot.py --capture --snapshot 01_data/experiment.json --limit 2

# build the course index, then query it
python 02_src/curriculum_ingest.py --curriculum 01_data/curriculum --db ./vectorstore
python 02_src/curriculum_ingest.py --db ./vectorstore --query "chunking" --week 2

# monitoring and clustering
python 02_src/monitoring_github.py --days 30
python 02_src/monitoring_rss.py --days 30 [--secondary]
python 02_src/clustering.py --signals 01_data/signals.json --check

# evaluation
python 04_eval/run_eval.py --repeats 3 --out 04_eval/results/<name>.json
python 04_eval/compare.py <baseline.json> <after.json>
```

C Repository structure

```

01_data/           curriculum (Weeks 2-6, not in Git), signals*.json, demo_snapshot.json
02_src/           schemas, monitoring, clustering, curriculum_ingest, demo_snapshot
02_src/agents/    verification, curriculum, evaluation, recommendation,
                  companion, fix, tools, test_chain.py
02_src/tests/     test_verification.py
03_assets/        architecture diagram, logo, screenshots, presentation, this report
04_eval/          golden datasets, run_eval.py, compare.py, saved results
c_sync/           C-Sync, the Streamlit interface (app.py, ui*.py)
vectorstore/      local Chroma index, regenerated by curriculum_ingest.py

```

D Glossary

Signal	One item from a monitored source: a GitHub release or a blog post (RawSignal).
Primary / secondary source	Primary: the project's own words (release, official blog). Secondary: someone reporting on it (forum, news).
Cluster (event)	Signals judged to report the same underlying event; its distinct sources count as independent.
Confidence	Verification's 0–1 belief that the event is real, computed in code from checked facts.
Maturity / relevance	1–5 scores: how established the trend is, and how directly it hits what the course teaches.
Action tiers	<i>Keep watching, add optional content, update existing material, create a new lesson and investigate larger change</i> (not reachable by design).
Exact identifier match	A code identifier from the trend (e.g. RunnableWithMessageHistory) found verbatim in a slide or cell.
Trace	The recorded reasoning behind a recommendation: verification steps and tool results, curriculum queries and outcome.
Snapshot	A captured pipeline run saved to JSON and replayed by C-Sync with zero API calls.
Search failed	A curriculum search that was attempted but could not run. Never equivalent to “no match”.
Human in the loop	Every recommendation waits for an instructor's decision: approve, request changes or reject.